

MICROSERVICES COURSE

Session 6



Inter-Service Communication

Sync vs Async · OpenFeign · Design Choices · JWT Propagation



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

Monday

3:00 PM – 5:30 PM

Online Session

2.5 Hours

Phase 1

Session 6 of 29

Phase 1 — Foundation & Core Patterns



You are here — Session 6 of 8 in Phase 1. The platform survives failure. Today services finally TALK to each other.

This Session Is Not About OpenFeign

It is about how engineers DECIDE communication patterns.

The real skill is not writing `@FeignClient` — it is answering: "Why did I choose this pattern, and not another one?"

Sync or Async?

Does the caller need the answer to continue?

Feign, WebClient, or events?

Blocking call, reactive stream, or fire-and-forget?

What if it fails?

Retry? Compensate? Accept eventual consistency?

Sessions 6 & 7 are where you move from "how do I use the tool" to "how do I design the architecture."

Order Service Doesn't Actually Check Inventory

Right now: `createOrder()` goes straight to Payment. There is no Inventory Service. Nothing stops an order for a product with ZERO stock.

- 1 Customer orders 5 units of PROD-003
- 2 PROD-003 has 0 units in stock
- 3 Order Service has no way to know this
- 4 **Payment succeeds. Order is CONFIRMED.**
- 5 **Warehouse has nothing to ship. Customer is angry.**

Today: build Inventory Service, and connect it to Order Service — synchronously, with intent.

Sync or Async? Three Questions

Q1

Does the caller need the answer to CONTINUE?

YES → Sync (Feign/WebClient)
NO → Async (Kafka, Session 7)

Q2

Is the operation reversible if it fails?

YES → Sync is fine, retry
NO → Saga + compensation (S7)

Q3

Is eventual consistency acceptable?

NO → Sync (check stock NOW)
YES → Async (notify later)

Platform mapping:

Order → Inventory (check stock)

SYNC

Order → Notification (send email)

ASYNC

Order → Payment (charge card)

SYNC

RestTemplate vs OpenFeign vs WebClient

DESIGN CHOICE

Simple synchronous call, need it now?

→ OpenFeign

High-throughput reactive pipeline?

→ WebClient

Starting a new project today?

→ Avoid RestTemplate (deprecated path)

	RestTemplate	OpenFeign	WebClient
Style	Imperative	Declarative interface	Reactive / functional
Blocking	Yes	Yes	No
Status	Deprecated	Recommended (sync)	Recommended (reactive)

Building Inventory Service

Spring Initializr

- services/inventory-service
- Dependencies: Spring Web, Eureka Discovery Client, Spring Cloud Config, Actuator
- Port: 8084

StockItem.java – Stock domain model

```
public record StockItem(  
    String productId,  
    int availableQuantity,  
    int reservedQuantity) {  
    public boolean hasStock(int requested) {  
        return availableQuantity - reservedQuantity >= requested;  
    }  
}
```

StockCheckResponse.java – Stock Check DTO

```
public record StockCheckResponse(  
    String productId, int requestedQuantity, boolean available, int remainingStock  
) {}
```

Building Inventory Service

`InventoryService.java` – in-memory store

```
@Service
public class InventoryService {

    // In-memory store (PostgreSQL added as homework)
    // In-memory: PROD-001 (100 units), PROD-002 (5 units), PROD-003 (0 units – out of stock)
    private final Map<String, StockItem> stock = new ConcurrentHashMap<>(Map.of(
        "PROD-001", new StockItem("PROD-001", 100, 0),
        "PROD-002", new StockItem("PROD-002", 5, 0),
        "PROD-003", new StockItem("PROD-003", 0, 0) // out of stock
    ));

    public StockCheckResponse checkStock(String productId, int requestedQty) {
        StockItem item = stock.getOrDefault(productId,
            new StockItem(productId, 0, 0));
        boolean available = item.hasStock(requestedQty);
        return new StockCheckResponse(productId, requestedQty,
            available, item.availableQuantity() - item.reservedQuantity());
    }
}
```


Building Inventory Service

`InventoryController.java` — GET `/api/v1/inventory/check`

```
@RestController
@RequestMapping("/api/v1/inventory")
public class InventoryController {

    @Autowired
    private InventoryService inventoryService;

    @GetMapping("/check")
    public ResponseEntity<StockCheckResponse> checkStock(
        @RequestParam String productId,
        @RequestParam int quantity) {
        StockCheckResponse response = inventoryService.checkStock(productId, quantity);
        if (!response.available()) {
            return ResponseEntity.status(HttpStatus.CONFLICT).body(response);
        }
        return ResponseEntity.ok(response);
    }
}
```

Testing Inventory Service Directly

GET .../check?productId=PROD-001&quantity=5

200 OK

available: true, remainingStock: 100

GET .../check?productId=PROD-003&quantity=1

409 Conflict

available: false, remainingStock: 0

Inventory Service works correctly on its own. Next: Order Service calls it — synchronously, with OpenFeign.

OpenFeign Client - Order Service

Pom.xml — Add Feign dependency to Order Service

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

OrderServiceApplication.java — Enable Feign clients on main application

```
@SpringBootApplication
@EnableFeignClients // REQUIRED — scans for @FeignClient interfaces
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

Declaring the Feign Client - Order Service

`InventoryClient.java` — Define the Feign client interface

```
@FeignClient(  
    name = "INVENTORY-SERVICE", // must match Eureka service name  
    path = "/api/v1/inventory"  // base path for all methods  
)  
public interface InventoryClient {  
    @GetMapping("/check")  
    StockCheckResponse checkStock(  
        @RequestParam("productId") String productId,  
        @RequestParam("quantity")  int quantity  
    );  
}
```

```
// StockCheckResponse must be duplicated  
here (or use a shared library)  
// For this course: duplicate the record —  
shared libs discussed in Session 14  
public record StockCheckResponse(  
    String productId,  
    int requestedQuantity,  
    boolean available,  
    int remainingStock  
) {}
```



name = "INVENTORY-SERVICE" must match the Eureka service name EXACTLY. Feign resolves the address via Eureka — no hardcoded URL anywhere.

Calling Inventory Before Payment

`OrderService.java` — `createOrderAsync()` — Use Feign client

```
@Autowired
private InventoryClient inventoryClient; // injected by Feign

@Bulkhead(...) @TimeLimiter(...) @CircuitBreaker(...) @Retry(...)
public CompletableFuture<OrderResponse> createOrderAsync(OrderRequest request) {
    return CompletableFuture.supplyAsync(() -> {
        // Step 1: Check inventory BEFORE payment
        StockCheckResponse stock = inventoryClient.checkStock(
            request.getProductId(), request.getQuantity());
        if (!stock.available()) {
            return new OrderResponse("REJECTED",
                "Insufficient stock: only " + stock.remainingStock() + " available");
        }
        // Step 2: Process payment (only if stock is OK)
        PaymentResponse payment = paymentService.processPayment(
            new PaymentRequest(request.getAmount()));
        return new OrderResponse("CONFIRMED", payment.getTransactionId());
    });
}
```

The Full Chain — Order Checks Inventory First

POST /api/orders

```
productId: PROD-003, quantity: 1
```



Order Service → Inventory Service

```
Feign call via Eureka
```



Inventory responds

```
409 — out of stock
```



Order Service response

```
REJECTED — "Insufficient stock: only 0 available"
```

Try PROD-001 instead → CONFIRMED. Watch both services' logs to see the cross-service call.

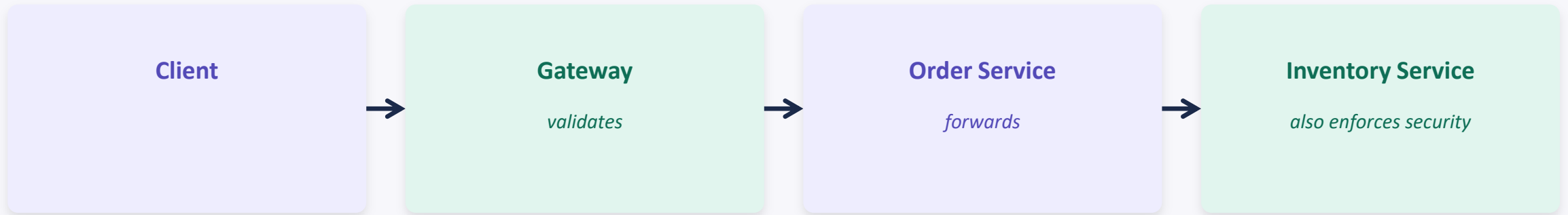
Translating HTTP Errors into Domain Exceptions

`InventoryErrorDecoder.java`

```
@Component
public class InventoryErrorDecoder implements ErrorDecoder {
    @Override
    public Exception decode(String methodKey, Response response) {
        return switch (response.status()) {
            case 409 -> new InsufficientStockException("Product out of stock");
            case 404 -> new ProductNotFoundException("Product not found");
            case 503 -> new ServiceUnavailableException("Inventory unavailable");
            default -> new FeignException.InternalServerError(
                "Unexpected error", null, null, null);
        };
    }
}
```

By default, Feign wraps ALL HTTP errors as generic `FeignException`. `ErrorDecoder` translates them into meaningful domain exceptions your business logic can handle.

JWT Propagation — The Trust Chain Continues



The JWT travels with the request through the ENTIRE service chain.

Gateway validates it once. Order Service reads it. Order forwards it to Inventory via a RequestInterceptor. Inventory can independently check who the user is.

Propagating the JWT to Inventory Service

FeignJwtInterceptor.java – Order Service

```
@Component
public class FeignJwtInterceptor implements RequestInterceptor {
    @Override
    public void apply(RequestTemplate template) {
        // Get JWT from the incoming request context
        var attrs = (ServletRequestAttributes) RequestContextHolder.getRequestAttributes();
        if (attrs != null) {
            String authHeader = attrs.getRequest().getHeader(HttpHeaders.AUTHORIZATION);
            if (authHeader != null) {
                template.header(HttpHeaders.AUTHORIZATION, authHeader);
            }
        }
    }
}

// Applies to ALL Feign clients in Order Service automatically
// Inventory Service receives the same JWT that the client sent to Order
```

WebClient — When Would You Reach for It?

Use WebClient when:

- Your service is built on WebFlux (reactive) throughout
- You need streaming responses (SSE, chunked data)
- You need fine-grained per-call control (custom timeouts, codecs)

In Our Platform Today:

OpenFeign for ALL service-to-service calls — blocking, simpler, matches our Spring MVC stack.

WebClient is used internally by Spring Cloud Gateway (it is WebFlux-based) — but we don't write it ourselves.

```
WebClient.builder()
    .baseUrl("lb://INVENTORY-SERVICE") // Eureka load balancing
    .build()
    .get()
    .uri("/api/v1/inventory/check?productId={id}&quantity={qty}", id, qty)
    .retrieve()
    .bodyToMono(StockCheckResponse.class)
    .block(); // WARNING: block() in reactive context = bad practice
```

WARNING: calling `.block()` inside a `WebClient` chain defeats its entire purpose — never do it in production code.

Lab 4A — Inventory Service + OpenFeign Order Call

1 Build Inventory Service

New service, port 8084, in-memory stock with 3 products

2 Add OpenFeign Client to Order Service

InventoryClient interface, check stock before payment

3 JWT Propagation

FeignJwtInterceptor forwards Authorization header

4 Write Unit Tests

Minimum 3 tests covering Feign call + error handling

Lab 4A Acceptance Criteria

- ✓ GET /api/v1/inventory/check?productId=PROD-001&quantity=5 → 200, available:true
- ✓ GET .../check?productId=PROD-003&quantity=1 → 409, available:false
- ✓ POST /api/orders with PROD-003 → REJECTED (insufficient stock)
- ✓ POST /api/orders with PROD-001 → CONFIRMED (stock checked + payment processed)
- ✓ JWT forwarded to Inventory Service — visible in Inventory's logs
- ✓ ErrorDecoder translates 409 into InsufficientStockException
- ✓ All 3 unit tests pass: mvn test
- ✓ Commit: session-06: add-inventory-service-and-feign-client

What We Added to the Platform Today



Capability Added

Sync Service-to-Service Communication

Services now talk via typed interfaces. Order checks real inventory before confirming — business rules cross service boundaries cleanly.



Services Added / Updated

- NEW: Inventory Service (8084)
- UPDATED: Order Service — InventoryClient (Feign)
- FeignJwtInterceptor — JWT propagation
- InventoryErrorDecoder — domain errors

- 🔗 Sessions 1-5: Config + Eureka + Gateway + Payment + Order (full resilience)
- 🔗 Session 6: Inventory Service + OpenFeign — first real service-to-service call

Config



Eureka



Product



Gateway



Sec.

Payment



Order



Resil.

Inventory



Notif.



S7



Daily Quiz

8 Questions · 10 Minutes

Topics: Sync vs Async Decision · OpenFeign vs WebClient · JWT Propagation · ErrorDecoder

Checkpoint Commit & Homework

Checkpoint — Definition of Done

- ☐ Inventory Service running on :8084
- ☐ Order rejects PROD-003 (no stock)
- ☐ Order confirms PROD-001 (has stock)
- ☐ JWT visible in Inventory logs
- ☐ Pushed: session-06: ...

Pre-Session 7 Reading (15 min)

Saga Pattern + Distributed Transactions
microservices.io/patterns/data/saga.html

Knowledge Check Questions for Session 7:

- What is Choreography Saga vs Orchestration Saga?
- What happens if Payment succeeds but Inventory fails?
- What is a Kafka consumer group?

Common Issues & Solutions



Feign client returns "no instances available"

Inventory Service must be UP and registered in Eureka before Order Service starts calling it



@FeignClient interface has no implementation

Missing @EnableFeignClients on the main application class



ErrorCode not catching errors

Register as @Component — Spring must discover it as a bean to wire into Feign



JWT not appearing in Inventory logs

Check FeignJwtInterceptor is a @Component and Authorization header exists on the original request



StockCheckResponse fields don't match

Record fields must match Inventory's response exactly — duplicate the record carefully



Session 6 Complete

Services now talk to each other — synchronously, with intent

Next: Session 7 — Distributed Transactions: Saga Pattern + Kafka · Wednesday, 3:00–5:30 PM, Online

[microservices-pro-course](#) · [microservices-pro-platform](#) · Dr. Sayed Baladoh